Name: Khushi Chhaproo
Div.: D20C
Roll No.: 07

Blockchain Experiment 9

Aim: To create a Token (ERC-20) in Remix IDE

Tools & Libraries Used:

- Remix IDE (LearnEth tutorials plugin, Solidity compiler, Deploy & Run) • MetaMask wallet (browser extension)
- 20lab (www.20lab.app), an online no-code token creator
- Test networks: Ethereum Sepolia (for 20lab) and Polygon Amoy, chain ID 80002 (for the Remix contract)
- Block explorers: Sepolia Etherscan and Amoy PolygonScan
- Solidity ^0.8.20

Theory:

Token in the blockchain ecosystem

A token is a digital asset that is created and managed by a smart contract on an existing blockchain. A coin such as ETH or POL is the native currency of its own blockchain, whereas a token lives inside a contract that simply keeps a table of which address owns how many units. Tokens can represent money (stablecoins), utility, voting rights in a project, loyalty points, or ownership of an item.

ERC-20 standard

ERC-20 (Ethereum Request for Comments 20) is the standard interface for fungible tokens, where every unit is identical and interchangeable with any other unit. Because every ERC-20 token exposes the same functions and events, wallets (such as MetaMask), exchanges and other contracts can work with any of them without special code. The standard defines six functions (totalSupply, balanceOf, transfer, approve, allowance, transferFrom), two events (Transfer, Approval) and the attributes name, symbol and decimals.

Decimals. The EVM has no floating-point numbers, so balances are stored as whole integers in the smallest unit. A token with decimals = 6 (like USDC) stores 1 token as 1,000,000 units, so minting 100 tokens means passing 100,000,000 to the contract.

Approve and transferFrom. A holder can allow another address (the spender) to spend a limited amount on their behalf by calling approve(spender, value). The spender then calls

transferFrom(owner, to, value), and the allowance decreases by the amount spent. This is how exchanges and DeFi contracts move tokens that the user has authorised.

Online token tool: 20lab

20lab is a web tool for creating tokens without writing code. The user connects a wallet, selects the token type (ERC-20), chooses the required features (for example pausable), validates the configuration and deploys; the deployment transaction is signed in the wallet. Testnet deployments are free of platform charges, so only test coins are spent as gas. Features chosen in the form are generated into a ready-made contract, which is why the tool is useful for quickly seeing what an ERC 20 token looks like before writing one by hand.

Smart contract used in the code part: TestUSDC

TestUSDC is a minimal, hand-written ERC-20 contract. It has the name "Test USD Coin", the symbol USDC and 6 decimals, and it implements all six ERC-20 functions and the Transfer and Approval events. It also has a public mint() function so that anyone can create test tokens for themselves (an extra Mint event is emitted as well). Every function checks its inputs with require(), for example it refuses transfers to the zero address, transfers larger than the balance and spending beyond the allowance. Because anyone can mint, this contract is for testing only and has no real value.

Tasks Performed:

Part A: LearnEth ERC-20 course in Remix IDE

1. Open Remix IDE (remix.ethereum.org). In the Plugin Manager activate the LearnEth plugin (or open the Remix link with LearnEth activated). The LearnEth icon appears in the left panel.
2. In LearnEth, open the tutorial list and select the Solidity ERC20 Token Course. The example files of the course are loaded into a workspace named LearnEth Tutorials.
3. Work through all 6 chapters of the course. In each chapter read the explanation, complete the exercise in the editor and press "Check Answer" until the chapter shows as passed, then move on with "Next".

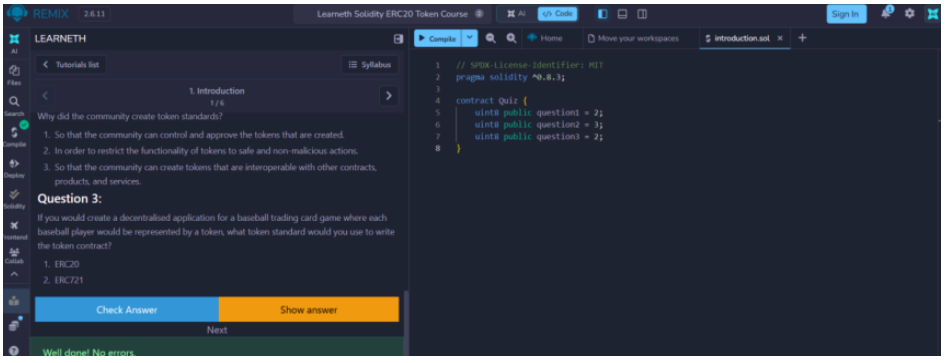


Fig. 1: Chapter 1 completed

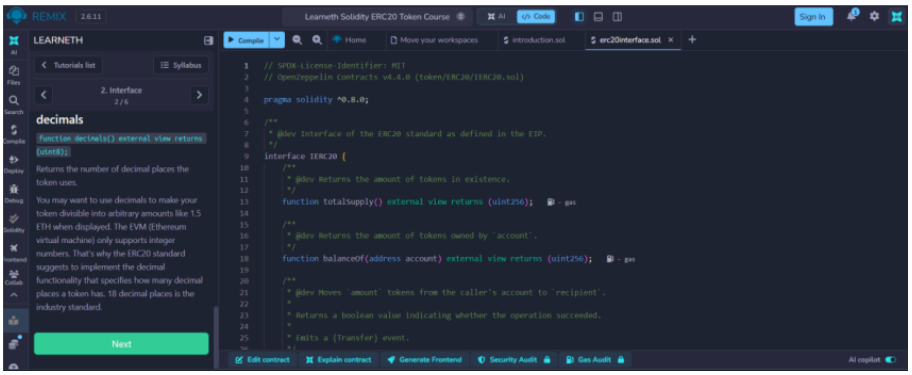


Fig. 2: Chapter 2 completed

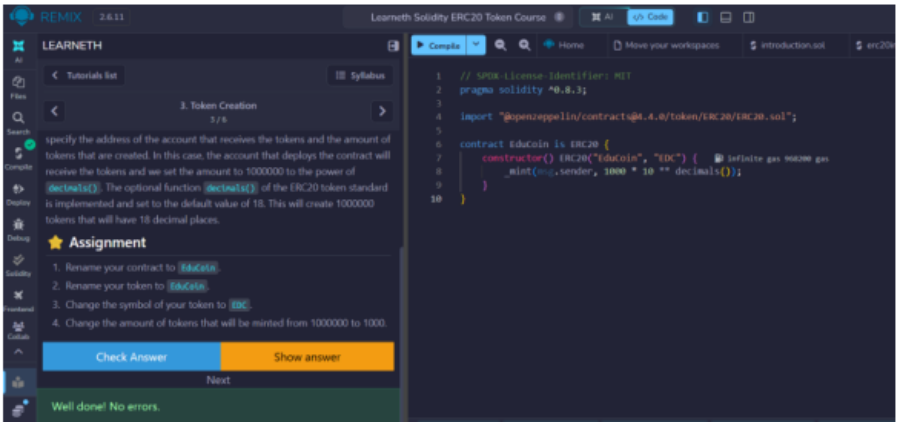


Fig. 3: Chapter 3 completed

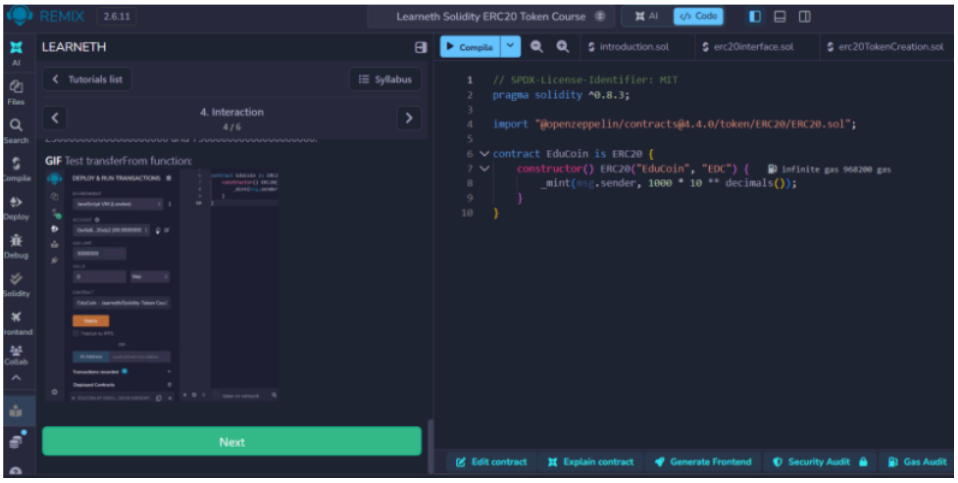


Fig. 4: Chapter 4 completed

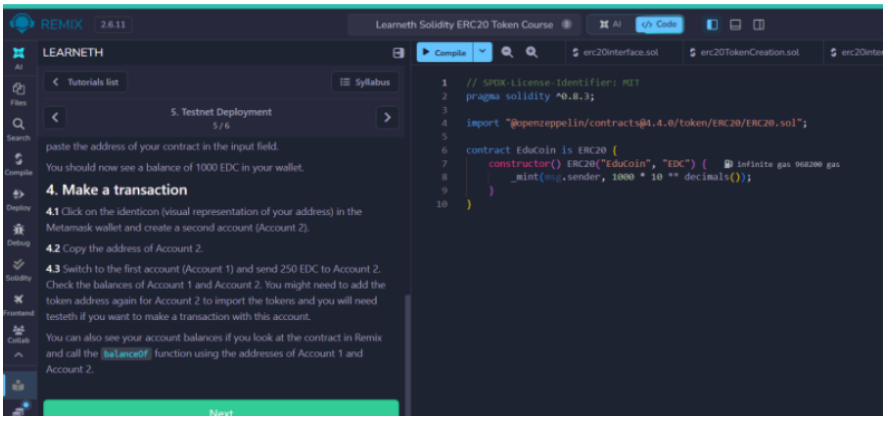


Fig. 5: Chapter 5 completed

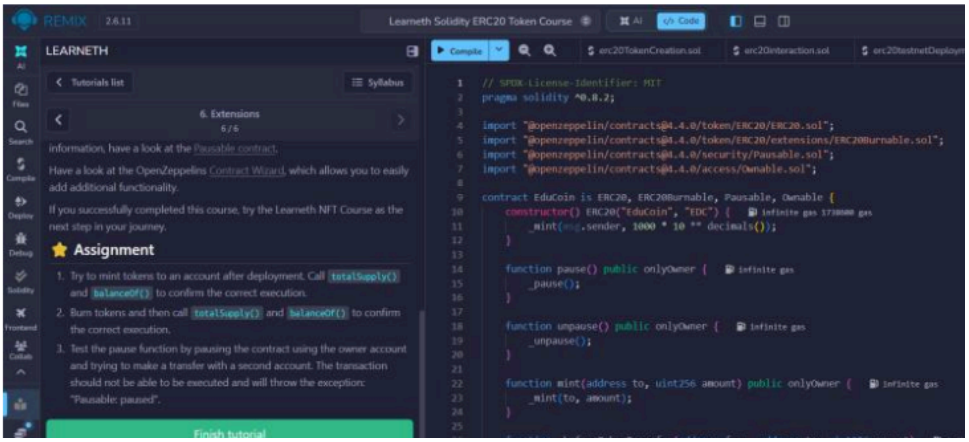


Fig. 6: Chapter 6 completed

Part B: Set up and fund the MetaMask wallet

The MetaMask wallet was created as in Experiment 8 (browser extension installed, new account created, Secret Recovery Phrase stored safely and never shown). The wallet was funded with test coins from faucets: Sepolia ETH (0.5 ETH per day) for the 20lab deployment and Polygon Amoy POL for the Remix deployment. The Google Cloud Web3 faucet can also be used for Sepolia.

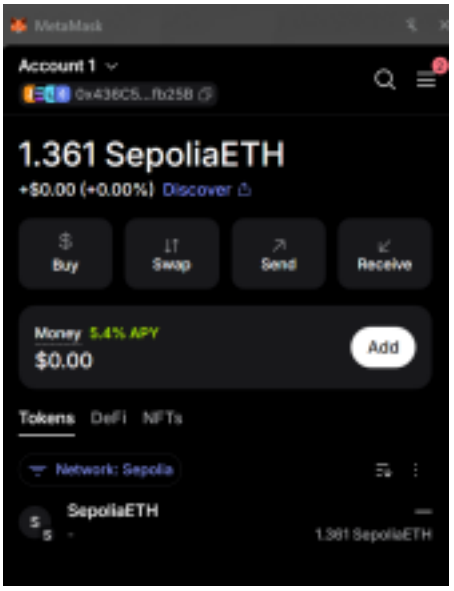


Fig. 7: Wallet funded with test coins on Sepolia and on Polygon Amoy

Part C: Create an ERC-20 token using the online tool 20lab

This part uses the Ethereum Sepolia testnet because 20lab offers Ethereum Sepolia among its free testnets. Before starting, switch MetaMask to the Sepolia network and check that it has test ETH.

1. Open www.20lab.app in the browser that has MetaMask installed.
2. Click the connect-wallet button, choose MetaMask and approve the connection request.
3. Select the token type ERC-20 and fill in the basic details the form asks for, for example token name (e.g. VESIT Test Token), symbol (e.g. VTT), decimals and initial supply.
4. Choose the features of the token. Keep the defaults, and optionally switch on Pausable so that the pause extension can be demonstrated for the viva question on Mintable / Burnable / Pausable.
5. Choose the network Ethereum Sepolia (testnet), validate the configuration and click Deploy.
6. MetaMask opens a transaction confirmation with the gas fee. Check that the network shows Sepolia and click Confirm.
7. After the transaction is mined the tool shows the token as deployed with its contract address. Copy the address.
8. In MetaMask open Tokens > Import tokens and paste the contract address. The symbol and decimals are filled in automatically. The initial supply appears in the deployer account's balance.
9. Open the contract address on the Sepolia block explorer (sepolia.etherscan.io) and note the creation transaction, the token name and symbol and the total supply.
10. Optionally open the owner dashboard of the tool and view the token settings.

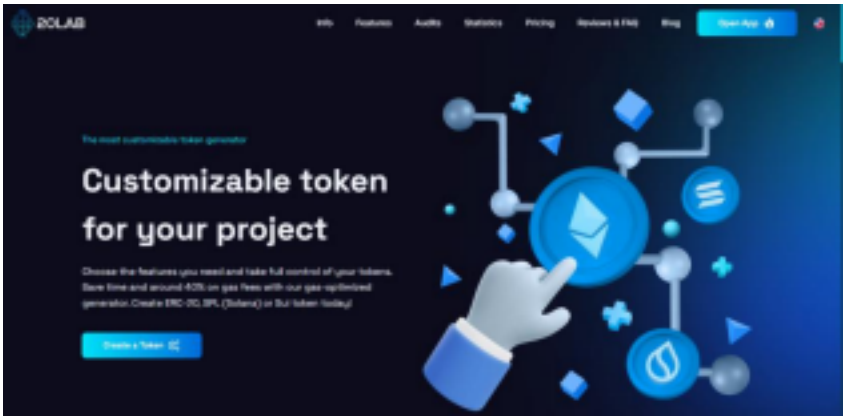


Fig. 8: 20lab home page

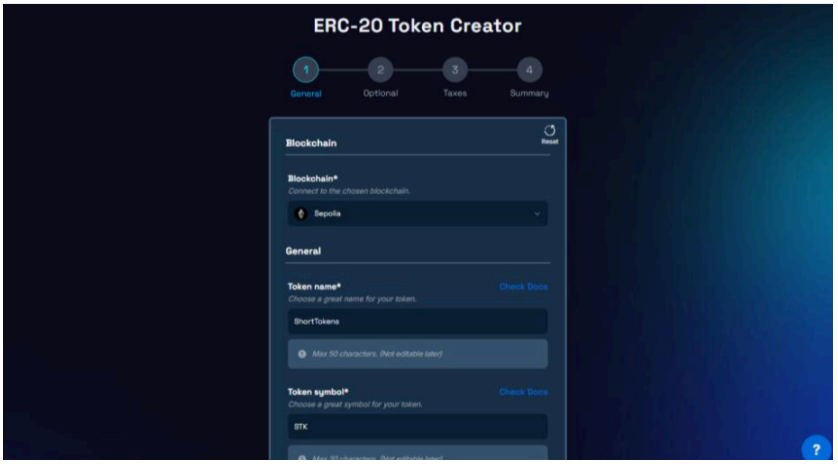


Fig. 9: ERC-20 token type selected and token details filled in

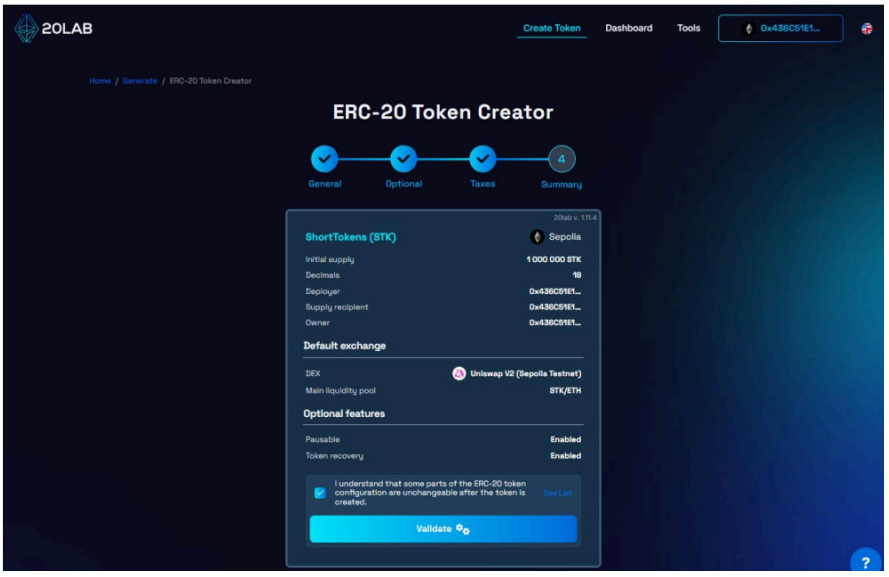


Fig. 10: Features of the token selected

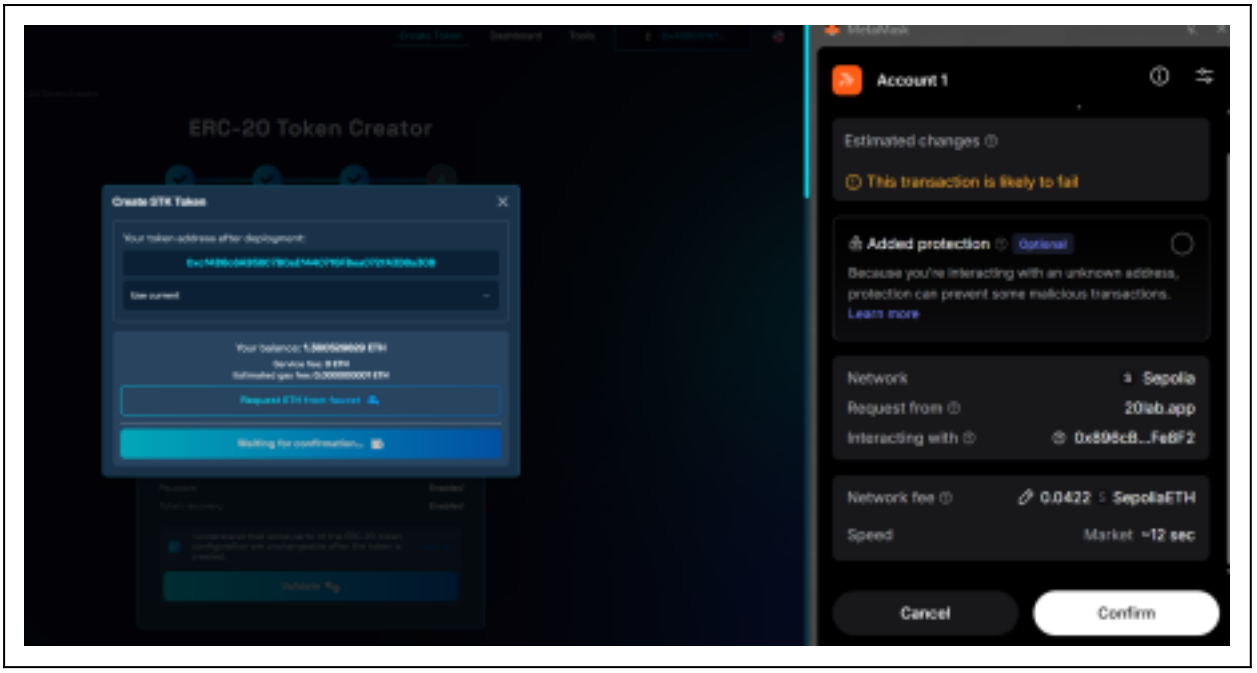


Fig. 11: MetaMask confirmation for the token deployment on Sepolia

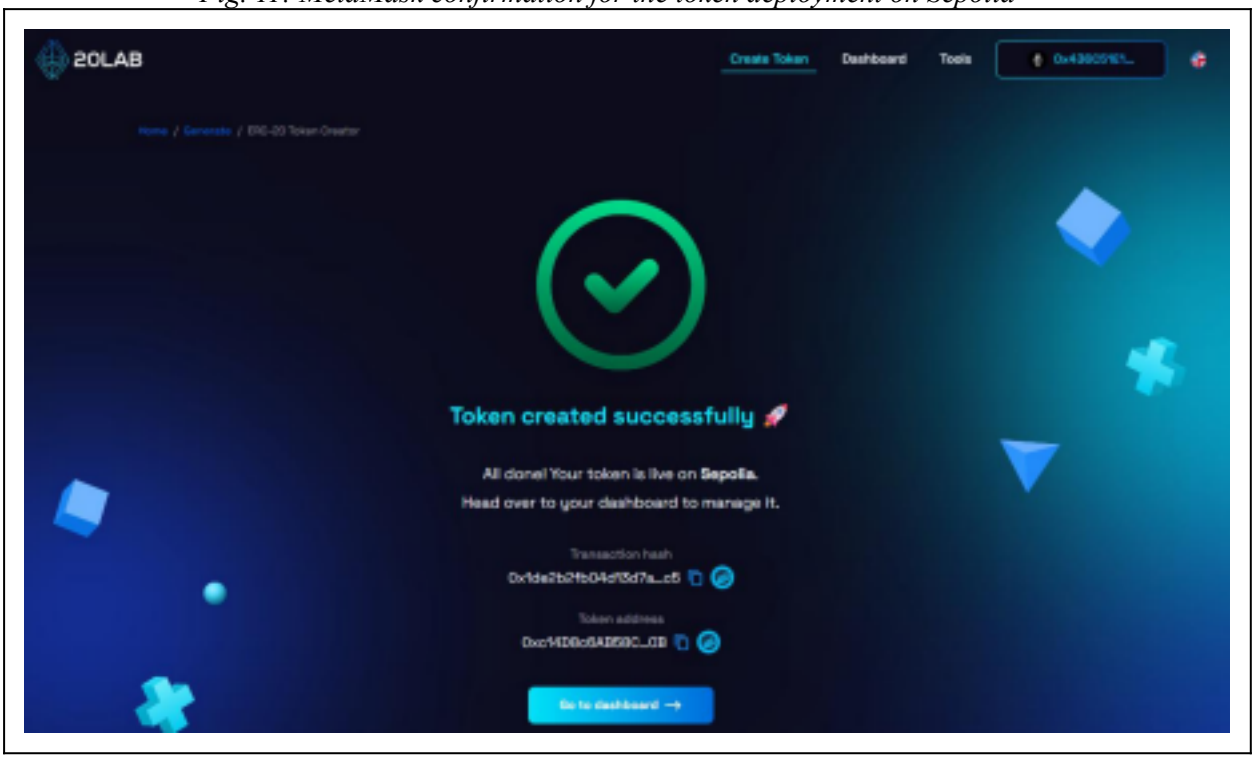


Fig. 12: Token deployed successfully with contract address shown

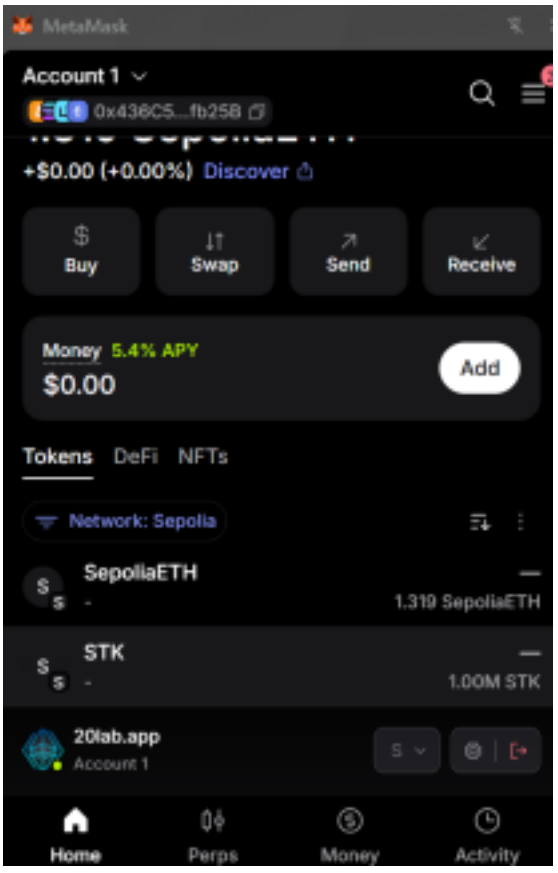


Fig. 13: Token imported into MetaMask showing the balance

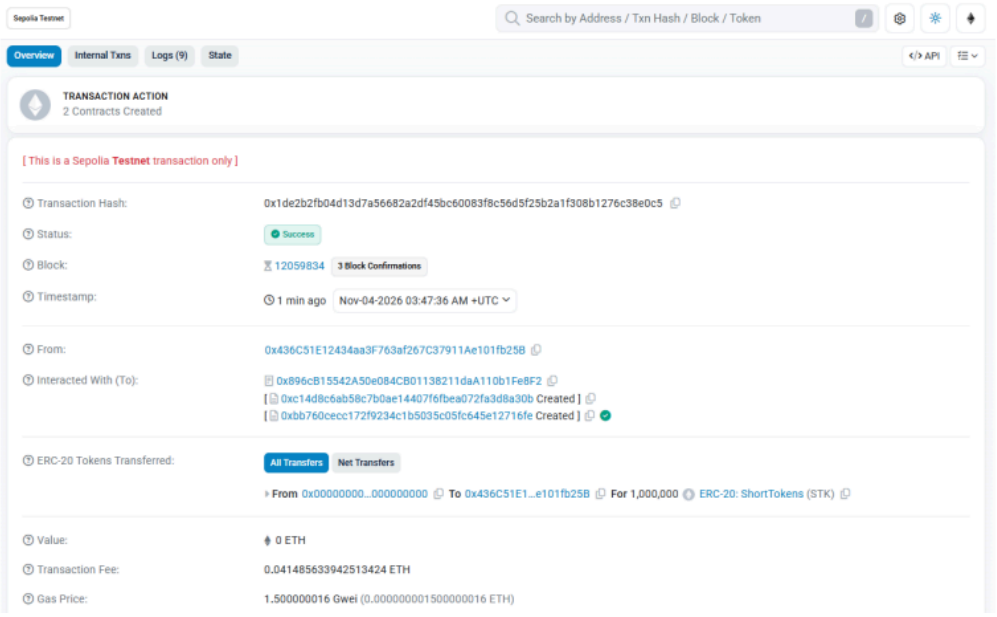


Fig. 14: Token contract page on the Sepolia block explorer

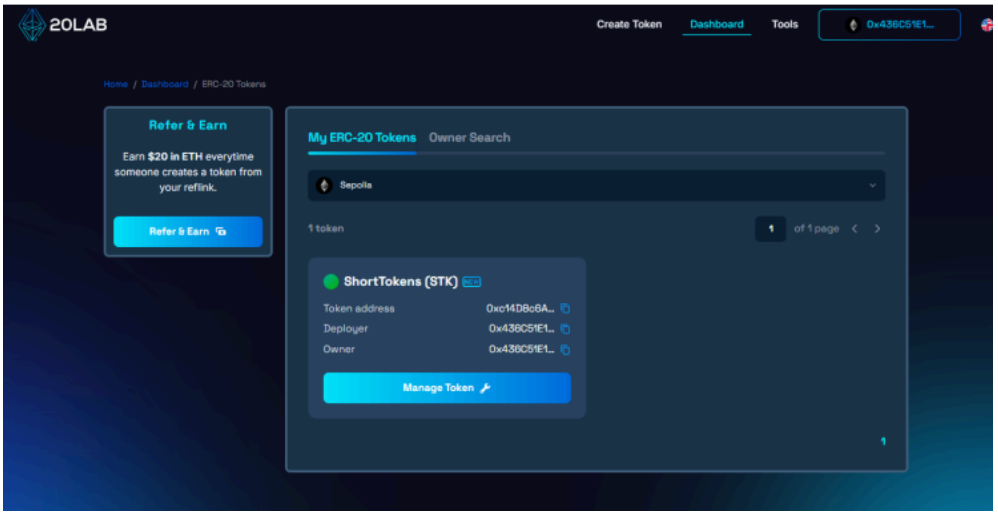


Fig. 15: Owner dashboard of the token on 20lab (optional)

Part D: Create an ERC-20 token using code in Remix IDE

The Solidity contract TestUSDC (listed in the Program section) was used. It was deployed on the Polygon Amoy testnet using MetaMask.

D1. Create and compile the contract

1. In the Remix File Explorer create the file TestUSDC.sol under contracts and paste the contract code.
2. Open the Solidity Compiler tab, select compiler version 0.8.20 or higher and click Compile TestUSDC.sol. A green tick on the compiler icon confirms a successful compilation.

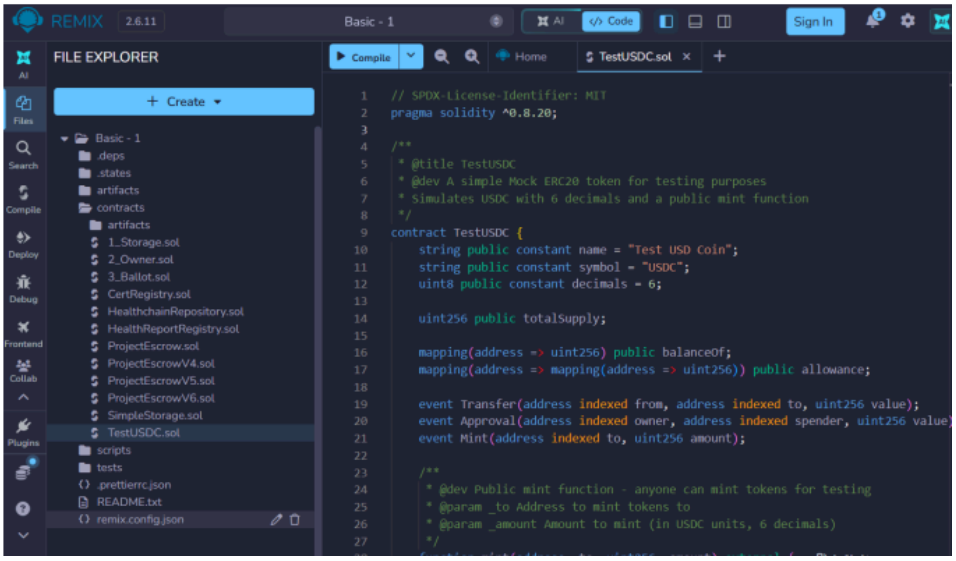


Fig. 16: TestUSDC.sol in the Remix editor

D2. Deploy to the testnet

1. In MetaMask select the Polygon Amoy network (chain ID 80002) and the account that has test POL.
2. In Remix open Deploy & Run Transactions and set the Environment to Injected Provider - MetaMask. Approve the connection. Remix shows the network (Amoy 80002) and the account balance.
3. Select the contract TestUSDC (the constructor takes no arguments) and click Deploy.
4. Confirm the transaction in MetaMask. If the deployment fails with an out-of-gas error, increase the gas limit in Remix and deploy again.
5. After mining, the contract appears under Deployed Contracts. Copy its address with the copy button.

D3. Check the transaction on the block explorer

The deployment transaction hash (from the Remix terminal or the MetaMask Activity tab) was searched on the Amoy block explorer (amoy.polygonscan.com). The page shows the status Success, the block number, the sender, the created contract address, the gas used and the transaction fee.

D4. Add the token to MetaMask

In MetaMask the token was added with Tokens > Import tokens by entering the TestUSDC contract address. The symbol USDC and the 6 decimals are picked up automatically. The balance shows after tokens are minted in step D5.

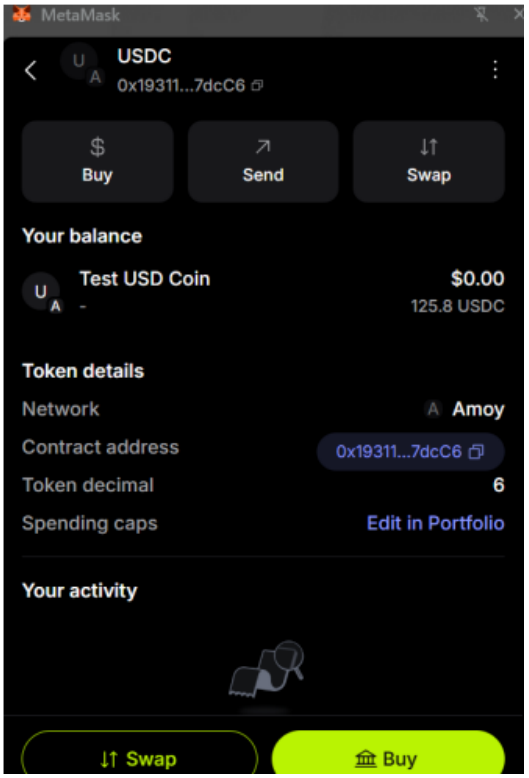


Fig. 17: TestUSDC token details in MetaMask (network Amoy, 6 decimals, balance)

D5. Interact with the token contract

The following calls were made from the Deployed Contracts panel in Remix. Amounts are entered in the smallest unit (6 decimals, so 1 USDC = 1,000,000). Account 1 is the deployer and Account 2 is a second MetaMask account (it needs a little test POL for gas). To send a call from Account 2, select Account 2 in MetaMask and Remix switches to it. Read-only calls need no gas and no confirmation; write calls need a MetaMask confirmation.

No.	Function (caller)	Input	Expected result
1	name, symbol, decimals, totalSupply (view)	none	Test USD Coin, USDC, 6, current total supply
2	mint (Account 1)	_to = Account 1 address, _amount = 100000000	Account 1 gets 100 USDC; totalSupply rises by 100 USDC; Mint and Transfer events
3	balanceOf (view)	Account 1 address	Balance increased by 100000000
4	transfer (Account 1)	_to = Account 2 address, _value = 10000000	Account 1 balance down by 10 USDC; Account 2 balance 10 USDC
5	approve (Account 1)	_spender = Account 2 address, _value = 5000000	Approval event; allowance(Account 1, Account 2) = 5000000
6	transferFrom (Account 2)	_from = Account 1, _to = Account 2, _value = 2000000	2 USDC moves from Account 1 to Account 2; allowance falls to 3000000
7	transfer (Account 1), failure test	_to = Account 2 address, _value larger than the balance	Transaction reverts: Insufficient balance
8	transferFrom (Account 2), failure test	_value larger than the remaining allowance	Transaction reverts: Insufficient allowance

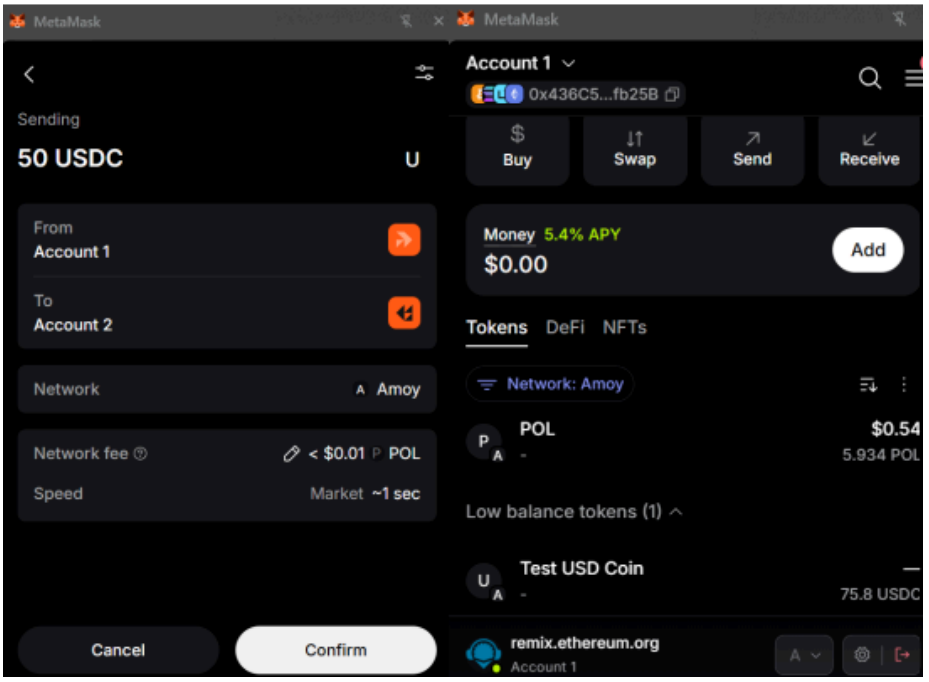


Fig. 18: approve transaction and allowance result and USDC balance of the account in MetaMask after all transactions

Program:

TestUSDC.sol

Compiler: Solidity 0.8.20 or higher. Deployed on the Polygon Amoy testnet.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

/**
 * @title TestUSDC
 * @dev A simple Mock ERC20 token for testing purposes
 * Simulates USDC with 6 decimals and a public mint function
 */
contract TestUSDC {
 string public constant name = "Test USD Coin";
 string public constant symbol = "USDC";
 uint8 public constant decimals = 6;

 uint256 public totalSupply;

 mapping(address => uint256) public balanceOf;
 mapping(address => mapping(address => uint256)) public allowance;

 event Transfer(address indexed from, address indexed to, uint256 value);
event Approval(address indexed owner, address indexed spender, uint256
value);  event Mint(address indexed to, uint256 amount);
```

```
 /**
 * @dev Public mint function - anyone can mint tokens for
testing  * @param _to Address to mint tokens to
 * @param _amount Amount to mint (in USDC units, 6
decimals)  */
 function mint(address _to, uint256 _amount) external {
 require(_to != address(0), "Cannot mint to zero
address");  require(_amount > 0, "Amount must be
greater than 0");

 totalSupply += _amount;
 balanceOf[_to] += _amount;

 emit Mint(_to, _amount);
 emit Transfer(address(0), _to, _amount);
 }

 /**
 * @dev Transfer tokens to a specified address
 * @param _to The address to transfer to
 * @param _value The amount to be transferred
 */
 function transfer(address _to, uint256 _value) external returns
(bool) {  require(_to != address(0), "Cannot transfer to zero
address");  require(balanceOf[msg.sender] >= _value, "Insufficient
balance");

 balanceOf[msg.sender] -= _value;
 balanceOf[_to] += _value;
 emit Transfer(msg.sender, _to, _value);
 return true;
 }

 /**
 * @dev Approve the passed address to spend the specified amount of
tokens  * @param _spender The address which will spend the funds
 * @param _value The amount of tokens to be spent
 */
 function approve(address _spender, uint256 _value) external returns
(bool) {  allowance[msg.sender][_spender] = _value;
 emit Approval(msg.sender, _spender, _value);
 return true;
 }

 /**
 * @dev Transfer tokens from one address to another
 * @param _from The address which you want to send tokens from
 * @param _to The address which you want to transfer to
```

```
 * @param _value The amount of tokens to be transferred
 */
 function transferFrom(address _from, address _to, uint256 _value) external returns
(bool) {  require(_to != address(0), "Cannot transfer to zero address");
 require(balanceOf[_from] >= _value, "Insufficient balance");
 require(allowance[_from][msg.sender] >= _value, "Insufficient allowance");

 balanceOf[_from] -= _value;
 balanceOf[_to] += _value;
 allowance[_from][msg.sender] -= _value;

 emit Transfer(_from, _to, _value);
 return true;
 }
}
```

Summary of the contract:

Member	**Description**
name, symbol, decimals	Constants: "Test USD Coin", "USDC" and 6.
totalSupply	Total number of token units in existence; increases when mint() is called.
balanceOf (mapping)	Balance of each address, in the smallest unit.
allowance (mapping)	Amount a spender is allowed to spend from an owner's balance.
mint(_to, _amount)	Creates new tokens for _to. Public, so anyone can call it (test use only). Emits Mint and Transfer.
transfer(_to, _value)	Moves tokens from the caller to _to after checking the zero address and the balance. Emits Transfer.
approve(_spender, _value)	Sets the amount _spender may spend on the caller's behalf. Emits Approval.

Member	**Description**
transferFrom(_from, _to, _value)	Lets an approved spender move tokens from _from to _to, reducing the allowance. Emits Transfer.

Inference

An ERC-20 token is a smart contract that keeps a table of balances. Any token that follows the standard functions and events (transfer, approve, transferFrom, Transfer,

Approval) shows up correctly in MetaMask and on the block explorer.

With 6 decimals, amounts are passed in base units: 100000000 is 100 USDC. The approve and transferFrom pair worked as an allowance, and the remaining allowance fell by exactly the amount spent.

Transfers above the balance and spending above the allowance reverted, so no tokens were lost. Write calls were mined and used gas, while read calls were free.

The 20lab tool deployed a token quickly with no code but only offered preset features. Writing TestUSDC in Remix gave full control, though its public mint() is only safe for a test token.

Conclusion:

The LearnEth ERC-20 course was completed in Remix IDE, a MetaMask wallet was funded with test coins, and ERC-20 tokens were created in two ways. A token was generated and deployed with the online tool 20lab on the Sepolia testnet, and a hand-written ERC-20 contract (TestUSDC) was compiled and deployed from Remix IDE on the Polygon Amoy testnet. The tokens were added to MetaMask, and minting, transfer, approval and transferFrom were performed and checked on the block explorer. The experiment gave an understanding of the ERC-20 standard and of the difference between ERC-20 and ERC-721 tokens and between the Mintable, Burnable and Pausable extensions.